

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №6.

“Разработка веб-приложения на Spring Boot. Основы тестирования”

Выполнил: студент группы БВТ2402

Скопа Михаил Вячеславович

Москва, 2026

Цель работы: Изучить основы тестирования в Java и Spring Boot-приложениях: научиться писать модульные тесты, проверять поведение отдельных компонентов приложения, изолировать зависимости с помощью моков-объектов, а также познакомиться с базовыми подходами к тестированию сервисного слоя и веб-приложений.

Задание для выполнения лабораторной работы

1. Напишите unit-тест для метода `getUserById()` в `UserService`.

```
@Test
void shouldSaveUserCorrectly() {
    UserDto dto = UserDto.builder().name("Иван").email("ivan@example.com").build();
    ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);

    userService.createUser(dto);

    verify(userRepository).save(userCaptor.capture());
    User savedUser = userCaptor.getValue();

    assertThat(savedUser.getName()).isEqualTo(expected: "Иван");
    assertThat(savedUser.getEmail()).isEqualTo(expected: "ivan@example.com");
}
}
```

✓ 1 test passed 1 test total, 1 sec 76 ms
"C:\Program Files\Java\jdk-17\bin\java.exe" ...

Инициализируется объект `UserDto` с заданными параметрами (`name`, `email`) с использованием паттерна `Builder`. Создается экземпляр `ArgumentCaptor` для перехвата объекта типа `User`, который будет передан в метод `save()`.

Вызывается метод `createUser` сервиса, который должен выполнить маппинг полей DTO в сущность и обратиться к слою репозитория.

С помощью `verify` проверяется факт вызова метода `userRepository.save()`. Используя `ArgumentCaptor`, перехватывается экземпляр сущности `User`, фактически переданный в репозиторий. Проводится проверка (`AssertJ`) на соответствие полей сохраненной сущности (`name` и `email`) исходным данным, переданным в `UserDto`.

2. Напишите unit-тест для метода `deleteUser()` и проверьте, что вызывается `userRepository.delete(...)`.

```
@Test
void shouldDeleteUserWhenUserExists() {
    Long userId = 1L;
    User mockUser = new User(); // Создаем пустой объект-заглушку

    when(userRepository.findById(userId)).thenReturn(Optional.of(mockUser));

    userService.deleteUser(userId);

    verify(userRepository, times(wantedNumberOfInvocations: 1)).delete(mockUser);
}
}
```

✓ 1 test passed 1 test total, 1 sec 48 ms

"C:\Program Files\Java\jdk-17\bin\java.exe" ...

Сначала с помощью имитации обращения к базе данных мы задаем условие, что при поиске по идентификатору возвращается существующий объект. Затем вызывается метод сервиса, отвечающий за удаление. В завершение выполняется проверка, подтверждающая, что сервис успешно передал найденный объект в репозиторий для выполнения операции удаления.

3. Реализуйте тест для метода `getNotificationById()` .

```
@Test
void shouldReturnNotificationWhenExists() {
    Long id = 1L;
    Notification mockNotification = new Notification();
    mockNotification.setId(id);

    when(notificationRepository.findById(id)).thenReturn(Optional.of(mockNotification));

    Notification result = notificationService.getNotificationById(id);
    |
    assertNotNull(result);
    assertEquals(id, result.getId());
    verify(notificationRepository, times(wantedNumberOfInvocations: 1)).findById(id);
}
```

Задается идентификатор (`id = 1L`) и создается объект-заглушка `mockNotification`, устанавливая ему этот ID. Затем настраивается поведение `notificationRepository`: указываем, что при вызове метода `findById(1L)` он должен вернуть не пустоту, а именно нашего подготовленного пользователя, обернутого в `Optional`.

Вызывается тестируемый метод `notificationService.getNotificationById(id)`. В этот момент сервис обращается к репозиторию, который, согласно настройке, возвращает заранее заданный объект уведомления.

`assertNotNull(result)` подтверждает, что сервис вернул объект, а не пустую ссылку.

- `assertEquals(id, result.getId())` проверяет, что возвращенное уведомление содержит именно тот ID, который мы запрашивали.
- `verify(notificationRepository, times(1)).findById(id)` удостоверяется, что сервис действительно обратился к репозиторию за поиском и сделал это ровно один раз.

4. Добавьте негативный тест для случая, когда уведомление не найдено.

```
@Test
void shouldThrowExceptionWhenNotFound() {
    Long id = 1L;
    when(notificationRepository.findById(id)).thenReturn( Optional.empty());

    assertThrows(RuntimeException.class, () -> notificationService.getNotificationById(id));
}
```

Проверка, что при поиске несуществующего Id выдаст ошибку `RuntimeException`

5. Создайте отдельный класс тестов для `AuthService` из лабораторной по Spring Security.

6. Используйте `verify(..., times(1))` для явной проверки количества ВЫЗОВОВ.

```
@Test
void shouldRegisterUserAndCallSaveOnce() {
    RegisterRequest request = new RegisterRequest( name: "Ivan", email: "ivan@test.com", password: "pass123"
    when(passwordEncoder.encode( rawPassword: "pass123")).thenReturn( value: "encodedPassword");

    authService.register(request);

    verify(userRepository, times( wantedNumberOfInvocations: 1)).save(any(User.class));
}

@Test
void shouldRegisterAdminAndCallSaveOnce() {
    RegisterRequest request = new RegisterRequest( name: "Admin", email: "admin@test.com", password: "adminPa
    when(passwordEncoder.encode( rawPassword: "adminPass")).thenReturn( value: "encodedPassword");

    authService.registerAdmin(request);

    verify(userRepository, times( wantedNumberOfInvocations: 1)).save(any(User.class));
}
```

✓ 2 tests passed 2 tests total, 1 sec 184 ms

"C:\Program Files\Java\jdk-17\bin\java.exe" ...

Мы создаем имитацию обращения к сервису для авторизации пользователя. Проверяем, что был создан нужный нам объект нужного типа и ровно 1 раз.

7. Попробуйте применить `spy()` к простому списку и проверьте вызов метода `add()`.

```
class SpyExampleTest {  
  
    @Test  
    void testSpyList() {  
        List<String> list = new ArrayList<>();  
        List<String> spyList = spy(list);  
  
        spyList.add("Hello");  
  
        verify(spyList, times(wantedNumberOfInvocations: 1)).add("Hello");  
  
        assertEquals(expected: 1, spyList.size());  
        assertEquals(expected: "Hello", spyList.get(0));  
    }  
}
```

Создается стандартный объект `ArrayList`, который затем «оборачивается» в `spy`. Это позволяет нам одновременно использовать реальные методы списка и отслеживать факт их вызова.

Вызывается метод `add("Hello")`. Так как это шпион, выполняется реальный код метода `ArrayList.add()`, что приводит к фактическому добавлению элемента в список.

С помощью метода `verify(..., times(1))` мы подтверждаем, что метод `add` был вызван ровно один раз с заданным аргументом. Это позволяет убедиться, что код действительно обратился к списку, как ожидалось.

Команды `assertEquals` подтверждают, что объект `spyList` действительно изменил свое внутреннее состояние: размер списка увеличился до 1, а добавленный элемент успешно сохранился.

8. Напишите простой `@WebMvcTest` для `NotificationController`.

```

@WebMvcTest(
    controllers = NotificationController.class,
    excludeFilters = @ComponentScan.Filter(
        type = FilterType.ASSIGNABLE_TYPE,
        classes = JwtAuthenticationFilter.class
    )
)
@AutoConfigureMockMvc(addFilters = false)
class NotificationControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @MockBean 2 usages
    private NotificationService notificationService;

    @Test
    void shouldGetNotificationById() throws Exception {
        Long id = 1L;
        Notification mockNotification = new Notification();
        mockNotification.setId(id);
        mockNotification.setTitle("Test Title");

        when(notificationService.getNotificationById(id)).thenReturn(mockNotification);

        mockMvc.perform(get(uriTemplate: "/notifications/{id}", id)
            .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("expression: $.id").value(id))
            .andExpect(jsonPath("expression: $.title").value(expectedValue: "Test Title"));

        verify(notificationService, times(wantedNumberOfInvocations: 1)).getNotificationById(id);
    }
}

```

Мы используем аннотацию `@WebMvcTest` для запуска только тех компонентов, которые относятся к веб-части программы, без загрузки всей базы данных или других тяжелых служб. Поскольку в приложении есть система защиты, мы отключаем проверку безопасности через `@AutoConfigureMockMvc(addFilters = false)`, чтобы сосредоточиться исключительно на логике самого контроллера.

Затем подменяем реальный сервис на его имитацию с помощью `@MockBean`. Так как тест работает изолированно и не имеет доступа к настоящей базе данных, эта аннотация создает объект-заглушку. В коде теста мы используем метод `when(...).thenReturn(...)`, чтобы заранее запрограммировать эту заглушку: если контроллер попросит уведомление с определенным идентификатором, она вернет наш заранее созданный тестовый объект.

После этого моделируем реальный запрос от пользователя с помощью объекта `MockMvc`. Отправляем запрос по адресу, соответствующему

получению уведомления, и анализируем полученный ответ. Тест проверяет несколько вещей: действительно ли сервер вернул статус «успешно» (`status().isOk()`), соответствуют ли данные в ответе (проверяются через `jsonPath`) тем, что мы задали, и обратился ли контроллер к сервису через `verify(...)`, чтобы убедиться, что взаимодействие произошло ровно один раз.

Вывод: Изучил основы тестирования в Java и Spring Boot-приложениях: научился писать модульные тесты, проверять поведение отдельных компонентов приложения, изолировать зависимости с помощью `mock`-объектов, а также познакомился с базовыми подходами к тестированию сервисного слоя и веб-приложений.